Multithreading in Java: Concepts, Examples, and Best Practices

Introduction

Multithreading is an important concept in Java that allows a program to do many things at the same time. Instead of running tasks one after another, a multithreaded program can run multiple tasks in parallel, making better use of system resources and improving performance.

In this comprehensive guide to multithreading in Java, we'll cover everything from basic thread creation to advanced concurrency control. You'll learn how to work with the `Thread` class, `Runnable` and `Callable` interfaces, and the modern `ExecutorService` framework. We'll explore the thread lifecycle, synchronization using synchronized, `wait()`/`notify()`, and thread-safe patterns for shared resources. This article also compares multithreading vs. parallelism, and outlines best practices and common mistakes to avoid. Whether you're new to Java concurrency or looking to deepen your expertise, this guide provides the knowledge and examples you need to write robust multithreaded applications.

▼

Key Takeaways

- Java supports multithreading natively through the `Thread` class, `Runnable` interface, and the `java.util.concurrent` package, enabling concurrent task execution within the same application.
- A Java thread goes through a defined lifecycle (New, Runnable, Running, Blocked/Waiting, and Terminated) managed by the JVM and OS.
- Threads can be created by extending the `Thread` class, implementing `Runnable`, or using lambda expressions and are started using the `start()` method.
- Thread management methods like `sleep()`, `join()`, `yield()`, and `setDaemon()` help control execution timing and coordination between threads.
- Synchronization is essential to avoid race conditions and ensure thread safety when accessing shared resources, using `synchronized`, `wait()`, `notify()`, or concurrent utilities.
- The `ExecutorService` framework is preferred over raw thread creation for efficient, scalable, and reusable task execution via thread pools.

- Following best practices like limiting shared state, shutting down executors properly, and using high-level concurrency tools ensures safer and more maintainable multithreaded applications.

▼

What is Multithreading?

In simple terms, multithreading means running multiple “threads” of execution within a single program. A thread is like a small, independent path of execution. Think of a thread as one person doing a task: if you have multiple threads, it’s like having several people working together on different parts of a job.

Java supports multithreading through built-in features. Each thread runs separately but shares the same memory space, which allows them to work together efficiently, but also means they need to coordinate carefully to avoid conflicts.

Why Use Multithreading in Java?

Multithreading offers several key advantages:

- **Improved Performance:** Threads can run in parallel on multi-core processors, making efficient use of system resources.
- **Responsive Applications:** Multithreading helps keep user interfaces responsive by offloading long-running tasks to background threads.
- **Resource Sharing:** Threads share memory and resources, enabling faster context switching and inter-thread communication.
- **Asynchronous Processing:** Tasks like file I/O, network requests, or database operations can run independently of the main application flow.

Real-World Use Cases of Multithreading

Multithreading is widely used in modern Java applications across many domains:

- **Web Servers:** Handle multiple client requests concurrently.
- **GUI Applications:** Keep the UI responsive during background operations.
- **Games:** Manage rendering, physics calculations, and input handling in parallel.
- **Real-Time Systems:** Perform time-sensitive tasks simultaneously.
- **Data Processing Pipelines:** Parallelize large-scale data computations and I/O operations.

▼

Multithreading vs. Parallel Computing

While multithreading and parallel computing are often used interchangeably in casual conversation, they refer to **distinct concepts** with different goals, characteristics, and use cases. Understanding the difference is crucial when designing performance-critical applications in Java, especially when working with concurrent or compute-intensive workloads.

What Is Parallel Computing?

Parallel computing is the process of dividing a large problem into smaller sub-problems that can be executed **simultaneously** across multiple processors or cores. The goal is to **reduce execution time** by making efficient use of hardware resources.

In Java, parallel computing is commonly used for:

- **CPU-bound tasks** (e.g., number crunching, simulations)
- **Data-parallel operations** (e.g., processing elements of a large array)
- **Batch processing** or **fork/join algorithms**

Java provides several tools for parallel computing, including:

- **Fork/Join Framework** (`java.util.concurrent.ForkJoinPool`)
- **Parallel streams** (`Stream.parallel()`)
- **Parallel arrays** in frameworks like Java Concurrency or third-party libraries

Key Differences: Multithreading versus Parallel Computing

The following table compares multithreading with parallel computing across various technical dimensions:

Feature	Multithreading	Parallel Computing
Primary Goal	Improve responsiveness and task coordination	Increase speed through simultaneous computation
Typical Use Case	I/O-bound or asynchronous tasks	CPU-bound or data-intensive workloads

Execution Model	Multiple threads, possibly interleaved on one core	Tasks distributed across multiple cores or processors
Concurrency vs. Parallelism	Primarily concurrency (tasks overlap in time)	True parallelism (tasks run at the same time)
Thread Communication	Often requires synchronization	Often independent tasks (less inter-thread communication)
Memory Access	Threads share memory	May share or partition memory
Java Tools & APIs	`Thread`, `ExecutorService`, `CompletableFuture`	`ForkJoinPool`, `parallelStream()`, and `ExecutorService` configured for CPU-bound tasks
Performance Bottlenecks	Thread contention, deadlocks, synchronization latency	Poor task decomposition, load imbalance
Scalability	Limited by synchronization and resource management	Limited by number of available CPU cores
Determinism	Often non-deterministic due to timing and order	Can be deterministic with proper design

When to Use Parallel Computing in Java

Use parallel computing techniques when your application performs **heavy, repetitive, or large-scale computations** that can be broken into **independent subtasks**. This includes:

- Image and video processing
- Mathematical simulations (e.g., physics, finance, statistics)
- Large dataset analysis
- Matrix or vector operations
- File parsing or transformation in batch jobs

▼

Understanding Java Threads

To work with multithreading in Java, it's important to understand what a **thread** is, how it works behind the scenes, and how it differs from a process. This section will explain the basics

of Java threads and how they differ from processes.

What is a Thread in Java?

A **thread** in Java is a lightweight unit of execution. It represents a single path of code that runs independently but shares memory and system resources with other threads in the same program. Threads are used to perform tasks in parallel, allowing your program to do multiple things at once.

Java makes it easy to work with threads using the built-in `Thread` class and other concurrency tools in the `java.util.concurrent` package. By default, every Java application starts with one main thread, which begins executing the `main()` method. From there, you can create additional threads to run tasks concurrently.

For example, you can create a thread to download a file while the main thread continues updating the user interface.

Thread vs. Process in Java

While threads and processes both allow tasks to run independently, it's important to understand the difference between a **thread** and a **process**:

Feature	Thread	Process
Definition	A smaller unit of a process	An independent program running in memory
Memory Sharing	Shares memory with other threads	Has its own separate memory space
Communication	Easier and faster (uses shared memory)	Slower (requires inter-process communication)
Overhead	Low	High
Example	Multiple tasks in a Java program	Running two different programs (e.g., a browser and a text editor)

In Java, when you run a program, the **Java Virtual Machine (JVM)** starts a process. Inside that process, the JVM can run multiple threads to perform different tasks.

Lifecycle of a Thread

A Java thread goes through several stages in its lifetime. These stages are managed by the JVM and the operating system.

Here are the main stages in the thread lifecycle:

- **New**: The thread is created but hasn't started yet. It's like assigning a worker to a task, but they haven't begun the work.

  ```
  Thread thread = new Thread();
  ```

- **Runnable**: The thread is ready to run and waiting for CPU time. It's like the worker is standing by, ready to work when given the chance.

  ```
  thread.start(); // Moves to Runnable
  ```

- **Running**: The thread is actively executing code. Only one thread per CPU core can be in this state at a time. From the JVM's perspective, a thread that is actively running is still in the `RUNNABLE` state. We separate them here conceptually to distinguish between a thread that is ready to run and one that is currently executing on a CPU.

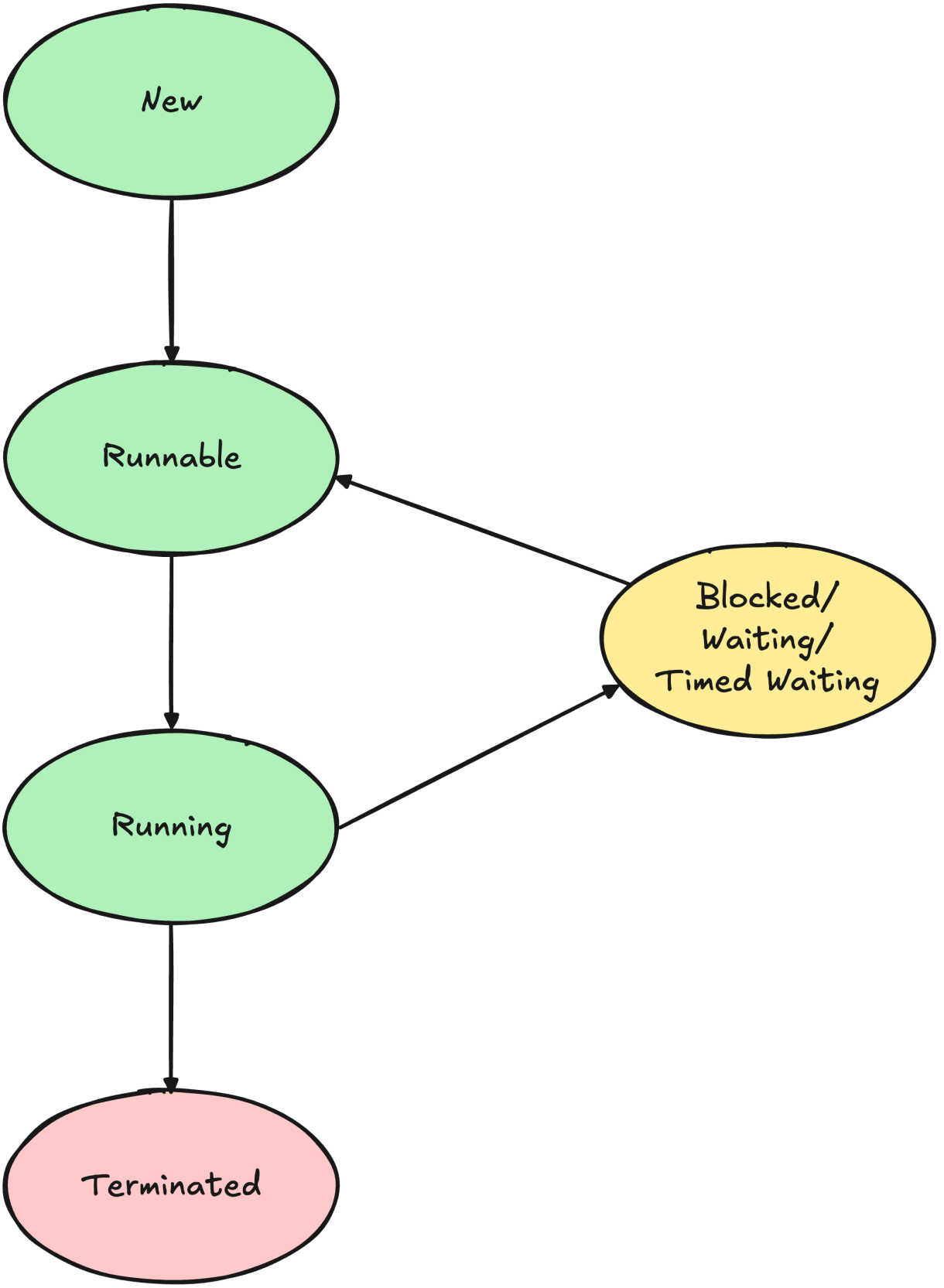
- **Blocked / Waiting / Timed Waiting**: The thread is temporarily inactive:

  - **Blocked**: Waiting for a resource (like a lock held by another thread)
  - **Waiting**: Waiting indefinitely for another thread's action (e.g., using `wait()`)
  - **Timed Waiting**: Waiting for a specified time (e.g., `sleep(1000)`)

    These states help threads pause without consuming CPU resources.

- **Terminated (Dead)**: The thread has finished executing or has been stopped. It cannot be restarted.

Here's a simple diagram to visualize the lifecycle:



Understanding the lifecycle helps you write better multithreaded programs and avoid common issues like deadlocks and resource contention.

▼

Creating Threads in Java

In Java, there are several ways to create and run threads. Each approach has its own use cases, and choosing the right one depends on how your application is structured. Here, we'll cover the three main ways to create threads:

1. Extending the `Thread` class
2. Implementing the `Runnable` interface
3. Using lambda expressions (Java 8 and later)

1. Extending the `Thread` Class

One of the simplest ways to create a thread in Java is by extending the built-in `Thread` class and overriding its `run()` method. This method contains the code that should run in the new thread.

Here's an example:

```
public class MyThread extends Thread {
    public void run() {
        System.out.println("Thread is running...");
    }

    public static void main(String[] args) {
        MyThread thread = new MyThread();
        thread.start(); // Start the thread
    }
}
```

The `run()` method defines the task the thread will perform. The `start()` method tells the JVM to start a new thread and execute `run()` in parallel with the main thread.

Use this approach when you don't need to extend another class (since Java supports single inheritance).

2. Implementing the `Runnable` Interface

A more flexible way to create threads is to implement the `Runnable` interface. This separates the task from the thread itself, which is useful when you want your class to extend another class.

Let's see an example:

```
public class MyRunnable implements Runnable {
    public void run() {
        System.out.println("Runnable thread is running...");
    }

    public static void main(String[] args) {
        Thread thread = new Thread(new MyRunnable());
        thread.start();
    }
}
```

This method is more flexible than extending `Thread`. It encourages better separation of concerns and allows you to reuse the same task with different threads

3. Using Lambda Expressions (Java 8+)

If you're using Java 8 or later, you can simplify thread creation using lambda expressions. This is ideal for short, one-off tasks where you don't want to write a full class.

Here's an example:

```
public class LambdaThread {
    public static void main(String[] args) {
        Thread thread = new Thread(() -> {
            System.out.println("Thread running with lambda!");
        });
        thread.start();
    }
}
```

This method has the following advantages:

- Cleaner and more concise code
- Useful for quick background tasks
- Ideal when using thread pools or executor services

Thread Creation Comparison

Method	Inheritance Used	Reusability	Concise	Best For
Extend `Thread`	Yes	No	Moderate	Simple custom thread logic
Implement `Runnable`	No	Yes	Moderate	Reusable tasks, flexible design
Lambda Expression (Java 8+)	No	Yes	High	Quick and short-lived tasks

▼

Thread Management and Control

Once you've created threads in Java, you need ways to **control how and when they execute**. Java provides several built-in methods in the `Thread` class to manage thread behavior effectively.

We'll cover the following:

- Starting a thread
- Putting a thread to sleep
- Waiting for a thread to finish
- Yielding execution
- Setting priorities
- Creating daemon threads
- Stopping threads (the right way)

1. Starting a Thread with `start()`

You call the `start()` method to execute a thread. This tells the JVM to create a new thread and begin running its `run()` method in parallel with the current thread.

```
Thread thread = new Thread(() -> {
    System.out.println("Thread is running.");
});
thread.start(); // Starts the thread
```

Calling `start()` creates a new call stack and runs the thread's task independently. It's important not to call `run()` directly; doing so would execute the method in the current thread, not on a new one.

2. Pausing Execution with `sleep()`

The `Thread.sleep()` method pauses the current thread for a specific number of milliseconds. This is useful in many real-world scenarios. For instance, you might want to simulate a delay when polling a remote server, slow down animations or visual transitions in a game, or introduce a backoff strategy for retrying failed operations.

```
try {
    System.out.println("Sleeping for 2 seconds...");
    Thread.sleep(2000); // 2000 ms = 2 seconds
    System.out.println("Awake!");
} catch (InterruptedException e) {
    System.out.println("Thread interrupted during sleep.");
}
```

Note: Always handle `InterruptedException`, which may occur if another thread interrupts this one during sleep.

3. Waiting for a Thread to Finish with `join()`

The `join()` method allows one thread to wait for another to complete before continuing. This is especially useful when coordinating tasks between threads. For example, if you start a background task that performs a calculation, you might use `join()` to ensure the main thread waits for the result before moving forward. It also comes in handy when you need to ensure that worker threads finish before your application shuts down.

```
Thread worker = new Thread(() -> {
    System.out.println("Working...");
    try {
        Thread.sleep(3000);
    } catch (InterruptedException e) {
      // Restore the interrupted status
      Thread.currentThread().interrupt();
      e.printStackTrace();
    }
    System.out.println("Work complete.");
});

worker.start();

try {
    worker.join(); // Main thread waits for worker to finish
    System.out.println("Main thread resumes.");
} catch (InterruptedException e) {
    e.printStackTrace();
}
```

4. Yielding Execution with `yield()`

The `Thread.yield()` method suggests to the thread scheduler that the current thread is willing to pause and let other threads execute. However, this is just a hint. There's no guarantee that it will actually yield control. In practice, this method is rarely used, but it may be helpful in fine-tuning low-level thread behavior, such as in CPU-intensive computations that need cooperative multitasking.

```
Thread.yield();
```

Note: This method depends on the JVM and OS scheduler. It's rarely used in modern Java code.

5. Setting Thread Priority

Java allows you to assign priorities to threads using the `setPriority()` method, with values ranging from `Thread.MIN_PRIORITY` (1) to `Thread.MAX_PRIORITY` (10). The default is `Thread.NORM_PRIORITY` (5). Higher-priority threads are more likely to be scheduled first, but again, this depends on the JVM and underlying OS.

Thread priorities can be useful when you want certain tasks to run ahead of others. For example, in a game engine, you might give rendering threads higher priority than background data loading.

```
Thread thread = new Thread(() -> {});
thread.setPriority(Thread.MAX_PRIORITY); // Sets priority to 10
```

However, relying too heavily on priorities can lead to unpredictable behavior and should be done with care.

6. Daemon Threads

Daemon threads are background threads that don't prevent the Java Virtual Machine from exiting once all user (non-daemon) threads have completed. These are commonly used for background services that should not block application termination.

For example, a thread that performs periodic logging, monitoring, or cleanup operations is a good candidate for a daemon. If the rest of the application finishes, you usually don't want these background tasks to keep the program running.

```
Thread daemon = new Thread(() -> {
    while (!Thread.currentThread().isInterrupted()) { // Check for interrupt
        System.out.println("Background task...");
        try {
            Thread.sleep(1000);
        } catch (InterruptedException e) {
            // Exit the loop if interrupted
            Thread.currentThread().interrupt(); // Restore status
            break;
        }
    }
```

```
    System.out.println("Daemon thread stopping.");
});
```

Stopping a Thread (The Safe Way)

Java's `Thread.stop()` method is deprecated because it can cause threads to terminate in the middle of critical operations, leaving shared data in an inconsistent state. Instead, the recommended way to stop a thread is to use interruption or a volatile flag.

For instance, in a long-running task, you can check whether the thread has been interrupted and exit gracefully if it has. This is especially important when your thread is working with shared resources or performing critical operations like file writes or database updates.

```
Thread thread = new Thread(() -> {
    while (!Thread.currentThread().isInterrupted()) {
        // Perform task
    }
    System.out.println("Thread interrupted and stopping.");
});

thread.start();
thread.interrupt(); // Gracefully request stop
```

▼

Synchronization and Concurrency Control

When multiple threads access shared data or resources, problems can arise if their actions are not coordinated. Without proper control, threads might interfere with each other, leading to incorrect results, crashes, or unpredictable behavior.

Why Synchronization Is Necessary

In multithreaded programs, threads often **share variables or objects** in memory. For example, two threads may try to update the same bank account balance at the same time. If both threads read the value before either writes it, the final result could be incorrect. This is known as a **race condition**.

Without proper control, even basic operations like incrementing a counter (`count++`) can lead to inconsistent data when accessed by multiple threads concurrently.

What Is a Race Condition?

A **race condition** occurs when two or more threads access shared data at the same time, and the outcome depends on the order of execution. These bugs can be hard to detect and reproduce because they depend on timing.

```
public class CounterExample {
    static int count = 0;

    public static void main(String[] args) throws InterruptedException {
        Thread t1 = new Thread(() -> {
            for (int i = 0; i < 10000; i++) count++;
        });

        Thread t2 = new Thread(() -> {
            for (int i = 0; i < 10000; i++) count++;
        });

        t1.start();
        t2.start();
        t1.join();
        t2.join();

        System.out.println("Final count: " + count); // Output may vary!
    }
}
```

Expected output is `20000`, but due to race conditions, you may see a smaller number.

Using the `synchronized` Keyword

To prevent race conditions, Java provides the `synchronized` keyword. It allows only one thread at a time to execute a block of code or method that accesses shared resources.

Synchronized Method:

```
public synchronized void increment() {
    count++;
}
```

Synchronized Block:

```
public void increment() {
    synchronized (this) {
        count++;
    }
}
```

In both cases, Java places a **lock** (or monitor) on the object, allowing only one thread to enter the synchronized code at a time.

Static Synchronization

If the shared data is static (class-level), you need to synchronize on the class itself:

```
public static synchronized void staticIncrement() {
    // synchronized at the class level
}
```

Or using a block:

```
public void staticIncrement() {
    synchronized (CounterExample.class) {
        count++;
    }
}
```

Synchronizing Only Critical Sections

It's best to synchronize only the smallest possible portion of code that needs protection, called the **critical section**, to reduce contention and improve performance.

```
public void updateData() {
    // non-critical code
    synchronized (this) {
        // update shared data
    }
    // more non-critical code
}
```

Thread Safety and Immutability

Another way to ensure thread safety is by designing **immutable objects**, objects whose state cannot change after creation. If threads only read from immutable objects, no synchronization is needed.

For mutable shared state, you can also use thread-safe classes like:

- `AtomicInteger`, `AtomicBoolean` (from `java.util.concurrent.atomic`)
- `ConcurrentHashMap`
- `Collections.synchronizedList()`

Avoiding Deadlocks

A **deadlock** occurs when two or more threads are waiting for each other to release locks, and none can proceed. This usually happens when multiple locks are acquired in an inconsistent order.

```
synchronized (resourceA) {
    synchronized (resourceB) {
        // do something
    }
}

// In another thread:
synchronized (resourceB) {
```

```
        synchronized (resourceA) {
            // do something else
        }
    }
```

To prevent deadlock:

- Always acquire locks in the same order.
- Avoid nested synchronized blocks if possible.
- Use timeout-based locking (`tryLock()` with a timeout) via `ReentrantLock`.

▼

Advanced Multithreading Concepts

As your applications grow in complexity, simply creating and synchronizing threads isn't enough. You'll need advanced tools and patterns for **inter-thread communication**, **memory visibility**, and **coordinated execution**. Java provides several powerful constructs for this, including `wait()`, `notify()`, `volatile`, and more.

Thread Communication with `wait()` and `notify()`

Sometimes threads need to **coordinate their actions**; one thread waits for a signal from another. For example, in a producer-consumer setup, the producer adds items to a queue and the consumer waits until items are available.

Java provides three methods to support this:

- `wait()`: Causes the current thread to release the lock and wait until another thread calls `notify()` or `notifyAll()`.
- `notify()`: Wakes up a single waiting thread.
- `notifyAll()`: Wakes up all waiting threads.

Example:

```
class SharedData {
    private boolean available = false;

    public synchronized void produce() throws InterruptedException {
```

```
        while (available) {
            wait(); // Wait until item is consumed
        }
        System.out.println("Producing item...");
        available = true;
        notify(); // Notify waiting consumer
    }

    public synchronized void consume() throws InterruptedException {
        while (!available) {
            wait(); // Wait until item is produced
        }
        System.out.println("Consuming item...");
        available = false;
        notify(); // Notify waiting producer
    }
}
```

Always call `wait()` and `notify()` inside a synchronized block or method. Also use a loop to recheck the condition after waking up—this guards against **spurious wakeups**.

The `volatile` Keyword

By default, changes made by one thread may not be immediately visible to others due to CPU caching or compiler optimizations. The `volatile` keyword tells the JVM to **always read the latest value of a variable from main memory**.

Example:

```
class FlagExample {
    private volatile boolean running = true;

    public void stop() {
        running = false;
    }

    public void run() {
```

```
        while (running) {
            // do work
        }
    }
}
```

Use `volatile` for flags and simple state indicators. It doesn't provide atomicity for compound actions (like incrementing), but it does ensure **visibility**.

Using `ReentrantLock` for Fine-Grained Locking

Java's `ReentrantLock` class (from `java.util.concurrent.locks`) offers more control than the `synchronized` keyword. It supports features like:

- Try-locking with timeouts
- Interruptible locks
- Fair locking order

Example:

```
import java.util.concurrent.locks.ReentrantLock;

ReentrantLock lock = new ReentrantLock();

try {
    lock.lock();
    // critical section
} finally {
    lock.unlock(); // Always unlock in a finally block
}
```

Use `ReentrantLock` when you need advanced locking behavior or fine-grained control over access timing and fairness.

Deadlock Prevention Strategies

Deadlocks occur when multiple threads hold resources and wait for each other to release them. Preventing deadlocks requires a consistent and cautious design strategy.

Tips to avoid deadlocks:

- **Lock ordering:** Always acquire multiple locks in the same order across all threads.
- **Use `tryLock()` with timeout:** Avoid waiting forever by using timed lock attempts.
- **Limit nested locking:** The fewer nested locks, the lower the risk of deadlock.
- **Detect and recover:** In large systems, use deadlock detection algorithms or monitoring tools.

Thread-Safe Design Patterns

Some object-level patterns help avoid thread-safety issues without the need for explicit locking:

- **Immutable Objects:** These can't be modified after creation (e.g., `String`, `LocalDate`). No need to synchronize.
- **Thread-Local Storage (`ThreadLocal<T>`):** Provides a separate variable copy for each thread.
- **Worker Thread Pattern:** A thread (or thread pool) handles all tasks from a shared queue.
- **Producer-Consumer Pattern:** Decouples producing data from consuming it, often using `BlockingQueue`.

▼

Thread Pools and the Executor Framework

As your applications grow more complex, creating and managing individual threads manually becomes inefficient and error-prone. Java provides a high-level concurrency framework called the **Executor Framework** that simplifies thread management and improves performance using **thread pools**.

Why Use a Thread Pool?

Creating a new thread for every task in a Java application can quickly become inefficient and problematic, especially in high-concurrency environments. It adds overhead due to the cost of thread creation, increased memory usage, and the complexity of managing the thread lifecycle. Thread pools solve these issues by reusing a fixed number of threads to execute

multiple tasks. This approach improves performance, prevents the system from being overwhelmed by too many simultaneous threads, and centralizes thread management, making applications more scalable and easier to maintain.

Using `ExecutorService` to Run Tasks

Java provides the `ExecutorService` interface for managing thread pools. The most common way to get one is through the `Executors` utility class.

```
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class ThreadPoolExample {
    public static void main(String[] args) {
        ExecutorService executor = Executors.newFixedThreadPool(3); // 3 threads

        Runnable task = () -> {
            System.out.println("Running task in thread: " +
Thread.currentThread().getName());
        };

        for (int i = 0; i < 5; i++) {
            executor.submit(task); // Submits task to thread pool
        }

        executor.shutdown(); // Initiates graceful shutdown
    }
}
```

Using `Callable` and `Future` for Return Values

Unlike `Runnable`, which cannot return a result, `Callable` allows you to return a value and even throw checked exceptions.

```
import java.util.concurrent.*;
```

```
public class CallableExample {
    public static void main(String[] args) throws Exception {
        ExecutorService executor = Executors.newSingleThreadExecutor();

        Callable<String> task = () -> {
            Thread.sleep(1000);
            return "Task result";
        };

        Future<String> future = executor.submit(task);

        System.out.println("Waiting for result...");
        String result = future.get(); // Blocks until result is available
        System.out.println("Result: " + result);

        executor.shutdown();
    }
}
```

Types of Thread Pools in Executors

Java provides several predefined thread pool types for different use cases:

Factory Method	Description
`Executors.newFixedThreadPool(n)`	Pool with a fixed number of threads. Good for predictable workloads.
`Executors.newCachedThreadPool()`	Creates threads as needed and reuses idle threads. Best for short-lived asynchronous tasks.
`Executors.newSingleThreadExecutor()`	A single-threaded executor. Tasks are executed sequentially.
`Executors.newScheduledThreadPool(n)`	Allows scheduling tasks to run after a delay or at fixed intervals.

Properly Shutting Down Executors

Always shut down executors after use to free resources. You can do this using:

```
executor.shutdown(); // Initiates a graceful shutdown

if (!executor.awaitTermination(60, TimeUnit.SECONDS)) {
    executor.shutdownNow(); // Forces shutdown if tasks don't finish in time
}
```

Never forget to shut down the executor. Otherwise your application may hang due to non-daemon threads.

Executors versus Threads: When to Use What

Java provides two primary approaches to running concurrent tasks: manually managing threads with the `Thread` class, or using the more modern and scalable **Executor framework**. While both can execute code concurrently, they differ significantly in terms of abstraction, control, and suitability for real-world applications.

Raw Threads (`Thread` Class)

Using the `Thread` class gives you **low-level control** over concurrency. When you create a new thread with `new Thread()` and call `.start()`, you are explicitly requesting the JVM to create a brand-new operating system-level thread.

When to use:

- **For learning purposes:** Creating threads manually is a great way to understand how the thread lifecycle works: how threads start, run, block, and terminate.
- **For quick, one-time tasks:** If your program needs to perform a simple, short-lived task in the background and will never repeat that operation, manually creating a thread may be sufficient.
- **When you need low-level access:** In rare situations where you need to set specific thread properties (like thread name or priority), or interact with thread groups (which are now discouraged), raw threads give you that level of access.

Limitations:

- **Expensive to create and destroy:** Each thread creation incurs significant resource overhead. Repeatedly creating and discarding threads can severely impact performance.
- **No built-in limit or control:** You can accidentally create too many threads, leading to memory exhaustion (`OutOfMemoryError`) or system instability.
- **Complex lifecycle management:** There's no easy way to shut down or coordinate many threads, and handling results or exceptions is entirely manual.

ExecutorService (Executor Framework)

The **Executor framework**, introduced in Java 5, provides a **high-level abstraction** for managing threads. It decouples task submission from thread execution using a thread pool. Instead of creating a new thread for each task, the executor reuses existing threads, improving efficiency and scalability.

When to Use:

- **For general-purpose concurrent task execution:** The ExecutorService is the standard way to run multiple tasks in parallel in modern Java applications.
- **When performance matters:** Thread pools reduce overhead by reusing threads and limiting the number of active threads.
- **To improve scalability:** You can use fixed or dynamic thread pools (like `newFixedThreadPool()` or `newCachedThreadPool()`) to better manage concurrency under varying workloads.
- **When you need to return results:** Executors support `Callable` tasks and `Future` objects, which allow you to retrieve results or handle exceptions from asynchronous tasks.
- **For easier lifecycle control:** Executors provide built-in methods like `shutdown()` and `awaitTermination()` to gracefully terminate your thread pool.

The table below highlights the key differences between using raw threads and the ExecutorService, helping you decide which approach best fits your use case:

Use Case	**Raw Thread (`new Thread()`)**	**ExecutorService (`Executors`)**
Learning and experimentation	Yes	Yes
One-off, lightweight background task	Sometimes	Recommended

Real-world, production application	Not recommended	Preferred
Efficient thread reuse	Manual	Automatic
Handling return values or exceptions	Requires custom logic	Built-in via `Future`/`Callable`
Graceful shutdown of background work	Hard to coordinate	Easy with `shutdown()`
Managing many tasks concurrently	Inefficient and risky	Scalable and safe

▼

Common Mistakes in Java Multithreading

Despite Java's rich concurrency API, multithreaded programming remains complex and even small errors can lead to serious problems like race conditions, deadlocks, memory leaks, or unpredictable behavior. Below are some of the most common mistakes developers make when working with threads in Java, along with tips on how to avoid them.

Calling `run()` Instead of `start()`

A frequent beginner mistake is calling `.run()` directly on a thread instance, which runs the task in the **current thread** instead of starting a **new one**.

```
Thread t = new Thread(() -> doWork());
t.run(); // runs on the main thread
t.start(); // runs in a new thread
```

Not Handling `InterruptedException` Properly

Threads that perform blocking operations (e.g., `sleep()`, `wait()`, or `join()`) can be interrupted, but many developers ignore or swallow the `InterruptedException` without restoring the thread's interrupt status.

```
try {
    Thread.sleep(1000);
} catch (InterruptedException e) {
    // Don't just ignore it
}

try {
    Thread.sleep(1000);
} catch (InterruptedException e) {
    Thread.currentThread().interrupt(); // Restore interrupt status
}
```

Creating Too Many Threads

Spawning a new thread for every task may seem simple, but it quickly becomes unsustainable in real-world applications. This can cause:

- High CPU overhead from excessive context switching
- Memory exhaustion (`OutOfMemoryError`)
- System instability

Use `ExecutorService` with a bounded thread pool to control concurrency.

Ignoring Synchronization on Shared Resources

Accessing or modifying shared mutable variables across threads without proper synchronization leads to race conditions.

```
// Not thread-safe
counter++;
```

Use synchronized blocks, atomic variables (`AtomicInteger`), or thread-safe collections to protect shared data.

Deadlocks from Improper Lock Ordering

Acquiring multiple locks in inconsistent order can cause deadlocks, where threads wait on each other forever.

Always acquire locks in a consistent global order, or use `ReentrantLock.tryLock()` with timeouts.

Blocking the Main Thread

Running long or blocking operations (e.g., network calls, disk I/O) on the main thread can freeze the application, especially in GUI or server contexts.

Offload long-running tasks to worker threads or use `CompletableFuture`.

Forgetting to Shut Down Executor Services

Failing to shut down an `ExecutorService` can leave background threads alive and prevent the JVM from exiting.

```
ExecutorService executor = Executors.newFixedThreadPool(4);
// Forgetting this leads to memory/resource leaks
executor.shutdown();
```

Assuming Visibility Without `volatile` or Synchronization

Java's memory model doesn't guarantee that a change in one thread is visible to another unless the variable is declared `volatile` or accessed within a synchronized block.

```
// May loop forever if flag update isn't visible
while (!flag) {}
```

Use `volatile` or synchronized blocks to ensure visibility.

Overusing Synchronized Blocks

Excessive synchronization, especially around large blocks of code or frequently accessed methods, can lead to poor performance due to thread contention.

Keep synchronized sections as small and brief as possible; only lock the lines of code that are absolutely necessary.

Mixing `wait()`/`notify()` With Incorrect Locks

Calling `wait()` or `notify()` without holding the object's intrinsic lock (via `synchronized`) throws an `IllegalMonitorStateException`.

Invalid usage:

```
obj.wait();
```

Correct usage:

```
synchronized (obj) {
    obj.wait();
}
```

▼

Best Practices for Multithreading in Java

Writing multithreaded applications in Java requires more than just creating and starting threads. To build safe, scalable, and maintainable systems, developers need to follow sound engineering practices that address thread lifecycle management, synchronization, resource usage, and error handling.

Here are the most important and widely recommended **best practices** for multithreading in Java:

1. **Prefer ExecutorService Over Raw Threads**: Instead of manually creating threads with `new Thread()`, use the `ExecutorService` or `ForkJoinPool` for better resource management, thread reuse, and task scheduling. Executors decouple task submission from execution logic, improve performance, and simplify shutdown and error handling.
2. **Limit the Number of Active Threads**: Avoid creating more threads than the system can handle. Use fixed-size thread pools or bounded queues to keep concurrency within safe

limits. Excessive threads increase CPU context switching, memory usage, and can crash the JVM.

3. **Keep Synchronized Blocks Short and Specific**: Minimize the scope of `synchronized` blocks to only the critical section of code that accesses shared resources. Broad synchronization increases the risk of thread contention and reduces parallelism.

4. **Use Thread-Safe and Atomic Classes**: Use thread-safe collections (`ConcurrentHashMap`, `CopyOnWriteArrayList`) and atomic classes (`AtomicInteger`, `AtomicBoolean`) instead of manual synchronization when appropriate. These classes are optimized for concurrency and reduce boilerplate synchronization code.

5. **Avoid Deadlocks Through Lock Ordering or Timeouts**: Always acquire multiple locks in a consistent global order, or use timed locking mechanisms like `tryLock()` from `ReentrantLock`. Deadlocks can cause threads to hang indefinitely and are notoriously hard to debug.

6. **Properly Handle `InterruptedException`**: Always handle interruptions in thread code and propagate the interrupt status using `Thread.currentThread().interrupt()`. Ignoring interrupts can prevent threads from shutting down cleanly and make your application unresponsive.

7. **Gracefully Shut Down Executors**: Always shut down your thread pools using `shutdown()` or `shutdownNow()` followed by `awaitTermination()` to allow ongoing tasks to complete.

   ```
   executor.shutdown();
   executor.awaitTermination(30, TimeUnit.SECONDS);
   ```

   Unfinished or orphaned threads can prevent the JVM from exiting and lead to memory leaks.

8. **Name Your Threads for Easier Debugging**: Use custom thread factories or `Thread.setName()` to give meaningful names to threads, especially in thread pools.

   ```
   Executors.newFixedThreadPool(4, runnable -> {
       Thread t = new Thread(runnable);
       t.setName("Worker-" + t.getId());
   ```

```
        return t;
    });
```

Named threads help when analyzing logs, stack traces, or thread dumps in debugging tools like JVisualVM.

9. **Minimize Shared Mutable State**: Where possible, design threads to operate on independent data. Use immutable objects, thread-local variables, or data partitioning strategies to reduce shared state. Less shared state means fewer synchronization requirements and fewer chances of race conditions.

10. **Use Modern Concurrency Utilities**: Leverage the rich utilities in `java.util.concurrent`, such as:

    - `CountDownLatch`, `CyclicBarrier`, and `Semaphore` for thread coordination
    - `BlockingQueue` for producer-consumer patterns
    - `CompletableFuture` for asynchronous pipelines

    These abstractions reduce boilerplate and prevent many common concurrency bugs.

▼

Frequently Asked Questions (FAQs)

1. What is multithreading in Java?

Multithreading in Java is the process of running multiple threads (small, independent paths of execution) within a single program. This allows the application to perform several tasks concurrently, making better use of multi-core processors, improving performance, and keeping the application responsive. Threads share the same memory space, which allows them to work together efficiently.

2. What are the main ways to create a thread in Java?

There are three primary ways to create a thread in Java:

- **Extending the `Thread` class:** You create a new class that inherits from `Thread` and override its `run()` method.
- **Implementing the `Runnable` interface:** This is the preferred method. You create a class that implements `Runnable`, and then pass an instance of it to a `Thread`'s constructor.

This is more flexible as it allows your class to extend other classes.

- **Using Lambda Expressions (Java 8+):** For simple, one-off tasks, you can provide a lambda expression directly to the `Thread` constructor, making the code more concise.

3. What is the difference between calling `thread.start()` and `thread.run()`?

This is a critical distinction and a common mistake:

- **`thread.start()`:** This creates a new thread of execution and tells the Java Virtual Machine (JVM) to execute the `run()` method's code within that new thread. This is the correct way to achieve concurrency.
- **`thread.run()`:** This simply executes the code inside the `run()` method on the **current thread**, just like a regular method call. No new thread is created, and no concurrency is achieved.

4. What is a race condition and how do you prevent it in Java?

A race condition occurs when two or more threads access shared, mutable data at the same time, and the final outcome depends on the unpredictable order of their execution. This can lead to corrupted data. The primary way to prevent this is by using the `synchronized` keyword on a method or a block of code. This ensures that only one thread can execute that "critical section" at a time, protecting the shared data.

5. Why is it better to use an `ExecutorService` than creating threads manually?

Manually creating a new thread for every task (`new Thread()`) is inefficient, resource-intensive, and can lead to system instability if too many threads are created. The `ExecutorService` framework solves this by managing a **thread pool**—a collection of reusable worker threads. This improves performance, gives you control over the number of concurrent threads, and simplifies managing the lifecycle of tasks, especially those that return results (using `Callable` and `Future`).

6. What is a deadlock?

A deadlock is a situation where two or more threads are blocked forever, each waiting for a resource that the other thread holds. For example, Thread A acquires a lock on Resource 1 and waits for Resource 2, while Thread B holds Resource 2 and waits for Resource 1, resulting

in a deadlock. The most common strategy to prevent deadlocks is to ensure that all threads acquire multiple locks in a consistent, predefined order.

7. What is the purpose of the `thread.join()` method?

The `thread.join()` method forces the current thread to pause its execution and wait until the thread it is called on has finished its task and terminated. This is a key mechanism for coordinating threads, ensuring that a dependent task (e.g., in the main thread) only proceeds after its prerequisite background task is complete.

8. What's the difference between multithreading and parallel computing?

- **Multithreading** is a programming model for **concurrency**, which is about structuring a program to handle multiple tasks at the same time. On a single CPU core, this happens by interleaving tasks (context switching).
- **Parallel Computing** is the **true simultaneous execution** of tasks, which is only possible on hardware with multiple CPU cores.

In essence, multithreading is a technique you use in your code to *achieve* parallelism on capable hardware.

9. What is a good real-world example of multithreading in Java?

A great real-time example is a stock trading application:

- One thread is dedicated to receiving real-time price updates from the market data feed.
- Another thread handles the user interface, allowing the trader to view data and place orders without the UI freezing.
- When the trader places an order, a new worker thread is dispatched to process the transaction, communicate with the brokerage, and handle confirmation, all without interrupting the first two threads.
- A separate background thread might be running risk calculations or updating charts based on the incoming price data.

▼

Conclusion

In this article, we explored the core concepts of multithreading in Java—from thread lifecycle and creation methods to synchronization, inter-thread communication, and thread

coordination. We compared raw threads with the Executor framework, examined the differences between multithreading and parallel computing, and highlighted real-world use cases. Finally, we covered best practices and common pitfalls to help you write safer, more efficient concurrent code.

Mastering these concepts is essential for building responsive, scalable, and robust Java applications. With the right tools and practices, multithreading becomes a powerful capability in any Java developer's toolkit.

To deepen your understanding of multithreading, explore these hands-on tutorials:

- Thread Safety in Java — Learn how to prevent race conditions and ensure your code is safe when accessed by multiple threads.
- Java Thread Example — See practical examples of creating and running threads in Java, helping you get started with concurrent programming.
- Java Thread wait, notify and notifyAll Example — Understand how to coordinate threads using wait/notify mechanisms for effective inter-thread communication.
- Thread Life Cycle in Java - Thread States in Java — Explore the different states a thread can be in, and how threads transition between these states during their lifecycle.